TP N°1
Algorithme d'Euclide Étendu et Inversion Modulaire
Application au Chiffrement Affine

Master 1 Informatique
Sécurité et Cryptographie Appliquée

1 Introduction

L'objectif de ce TP est d'implémenter l'algorithme d'Euclide étendu afin de calculer le PGCD de deux entiers ainsi que les coefficients de Bézout. Cette implémentation permet ensuite de calculer l'inverse modulaire, élément fondamental en cryptographie, notamment dans le chiffrement affine.

2 Rappels Théoriques

2.1 PGCD et Identité de Bézout

Soient $a, b \in \mathbb{Z}$ non tous deux nuls. Le plus grand commun diviseur est défini par :

$$\gcd(a, b)$$

L'identité de Bézout affirme qu'il existe des entiers u et v tels que :

$$au + bv = \gcd(a, b)$$

L'algorithme d'Euclide étendu permet de déterminer explicitement ces coefficients.

2.2 Inversion Modulaire

Un entier a est inversible modulo n si :

$$\exists x \in \mathbb{Z} \quad tel \quad que \quad ax \equiv 1 \pmod{n}$$

L'inverse modulaire existe si et seulement si :

$$\gcd(a, n) = 1$$

Si l'algorithme d'Euclide étendu donne :

$$au + nv = 1$$

Alors :

$$a^{-1} \equiv u \pmod{n}$$

3 Implémentation

L'implémentation a été réalisée en Java en utilisant la classe `BigInteger` afin de manipuler des entiers de précision arbitraire.

3.1 Classe BigIntMath

Listing 1 – Classe BigIntMath : Algorithme d'Euclide étendu et inversion modulaire

```java
import java.math.BigInteger;

public class BigIntMath {

    public static BigInteger[] extendedGCD(BigInteger a, BigInteger
        b) {
        if (b.equals(BigInteger.ZERO)) {

            return new BigInteger[]{a, BigInteger.ONE, BigInteger.
               ZERO};
        } else {

            BigInteger[] vals = extendedGCD(b, a.mod(b));
            BigInteger d = vals[0];
            BigInteger u1 = vals[1];
            BigInteger v1 = vals[2];


            BigInteger u = v1;
            BigInteger v = u1.subtract(a.divide(b).multiply(v1));
            return new BigInteger[]{d, u, v};
        }
    }


    public static BigInteger modInverse(BigInteger a, BigInteger n)
        {
        BigInteger[] vals = extendedGCD(a, n);
        BigInteger d = vals[0];
        BigInteger u = vals[1];

        if (!d.equals(BigInteger.ONE)) {

            throw new ArithmeticException("Inverse␣modulaire␣
               inexistant");
        }


        return u.mod(n);
    }
}
```

Explications ligne par ligne :

- `if (b.equals(BigInteger.ZERO))` : Cas de base de la récursion. Le PGCD de a et 0 est a.
- `return new BigInteger[]a, BigInteger.ONE, BigInteger.ZERO` : On retourne les coefficients (1,0) pour Bézout.
- `BigInteger[] vals = extendedGCD(b, a.mod(b))` : Appel récursif de l'algorithme pour réduire le problème.
- `u = v1` et `v = u1.subtract(a.divide(b).multiply(v1))` : Mise à jour des coefficients selon la formule de l'algorithme étendu.
- `modInverse` utilise l'algorithme étendu et vérifie que gcd(a,n)=1 avant de renvoyer l'inverse modulo n.

3.2 Classe AffineCipher

Listing 2 – Classe AffineCipher : chiffrement et déchiffrement affine

```
import java.math.BigInteger;

public class AffineCipher {
    private BigInteger a;
    private BigInteger b;
    private BigInteger n;

    public AffineCipher(BigInteger a, BigInteger b, BigInteger n) {
        if (!BigIntMath.extendedGCD(a, n)[0].equals(BigInteger.ONE)
           ) {
            throw new IllegalArgumentException("a␣et␣n␣doivent␣
                tre ␣premiers␣entre␣eux");
        }
        this.a = a;
        this.b = b;
        this.n = n;
    }

    public BigInteger encrypt(BigInteger m) {
        return a.multiply(m).add(b).mod(n);
    }
    public BigInteger decrypt(BigInteger c) {
        BigInteger a_inv = BigIntMath.modInverse(a, n);
        return a_inv.multiply(c.subtract(b)).mod(n);
    }
}
```

Explications ligne par ligne :

- Le constructeur vérifie que $\gcd(a, n) = 1$ pour que l'inverse modulaire existe.
- `encrypt` applique la formule $(am + b) \mod n$ pour chiffrer.
- `decrypt` calcule a^{-1} avec `modInverse` puis applique la formule $(c - b) * a^{-1} \mod n$ pour retrouver le message original.

3.3 Classe Main

Listing 3 – Classe Main : Exemple d'utilisation du chiffrement affine

```
import java.math.BigInteger;

public class Main {
    public static void main(String[] args) {
        BigInteger a = BigInteger.valueOf(5);
        BigInteger b = BigInteger.valueOf(8);
        BigInteger n = BigInteger.valueOf(26);
        BigInteger m = BigInteger.valueOf(7);

        AffineCipher cipher = new AffineCipher(a, b, n);

        BigInteger c = cipher.encrypt(m);
        System.out.println("Message␣chiffr ␣:␣" + c);

        BigInteger decrypted = cipher.decrypt(c);
        System.out.println("Message␣d chiffr ␣:␣" + decrypted);
    }
}
```

Explications ligne par ligne :

- `BigInteger a = BigInteger.valueOf(5);` : Définition de la clé multiplicative du chiffrement affine.
- `BigInteger b = BigInteger.valueOf(8);` : Définition de la clé additive du chiffrement affine.
- `BigInteger n = BigInteger.valueOf(26);` : Définition du module (taille de l'alphabet).
- `BigInteger m = BigInteger.valueOf(7);` : Message clair à chiffrer, représenté par un entier.
- `AffineCipher cipher = new AffineCipher(a, b, n);` : Création de l'objet `AffineCipher` avec les paramètres a, b, n.
- `BigInteger c = cipher.encrypt(m);` : Chiffrement du message clair m.
- `System.out.println("Message chiffré : " + c);` : Affichage du message chiffré.
- `BigInteger decrypted = cipher.decrypt(c);` : Déchiffrement du message chiffré c pour retrouver le message original.
- `System.out.println("Message déchiffré : " + decrypted);` : Affichage du message déchiffré.

4 Résultats Expérimentaux

Pour les valeurs :

$$a = 5, \quad b = 8, \quad n = 26, \quad m = 7$$

On obtient :

$$E(7) = 17$$

Puis :

$$D(17) = 7$$

Ce qui vérifie expérimentalement :

$$D_{a,b}(E_{a,b}(m)) = m$$

consistency

5 Analyse

La réversibilité du chiffrement affine repose entièrement sur :

— La condition $\gcd(a, n) = 1$
— Le calcul correct de l'inverse modulaire
— L'algorithme d'Euclide étendu

Sans l'algorithme d'Euclide étendu, il serait impossible de calculer efficacement l'inverse modulaire pour de grands entiers.

6 Conclusion

Ce TP a permis de mettre en évidence le lien fondamental entre :

— Théorie des nombres
— Algorithme d'Euclide étendu
— Inversion modulaire
— Cryptographie classique (chiffrement affine)

L'utilisation d'entiers de précision arbitraire garantit la robustesse de l'implémentation et prépare à des applications cryptographiques plus avancées comme RSA.

1/1